

Self-Attention & Transformers — Essentials

Dense reference: only what's needed to understand the mechanism

1. The Problem

Mean-pooled embeddings (Stage 8 Part 1) discard word order entirely. Self-attention fixes this: for each token, dynamically weight and combine every other token, rather than averaging flatly.

2. Query, Key, Value

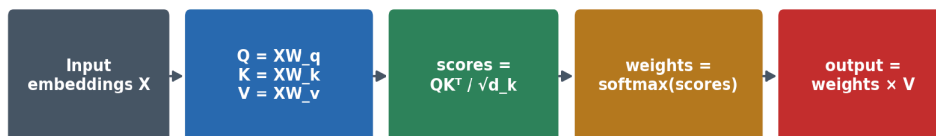
- **Query (Q)** — What this token is looking for.
- **Key (K)** — What each token advertises it contains.
- **Value (V)** — What each token actually passes along once deemed relevant.

Q, K, V are each just a learned linear transform of the same input embeddings (separate weight matrices W_q , W_k , W_v — ordinary learned weights, same as any other layer).

3. Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

- $Q \cdot K^T$ — dot product between every query and every key; entry $[i,j]$ = how relevant token j is to token i .
- $/ \sqrt{d_k}$ — prevents large dot products from saturating softmax as dimensionality grows.
- $\text{softmax}(\dots)$ — turns each row into weights summing to 1.
- $\cdot V$ — weighted sum of Values using those weights = the output for that token.



4. Multi-Head Attention

Runs several independent Q/K/V attention computations in parallel (each a "head"), each potentially learning a different kind of relationship between tokens. Outputs are concatenated and passed through one more linear layer. d_{model} is split evenly across heads (e.g. $d_{\text{model}}=512$, 8 heads $\rightarrow$ each head uses dimension 64).

5. Positional Encoding

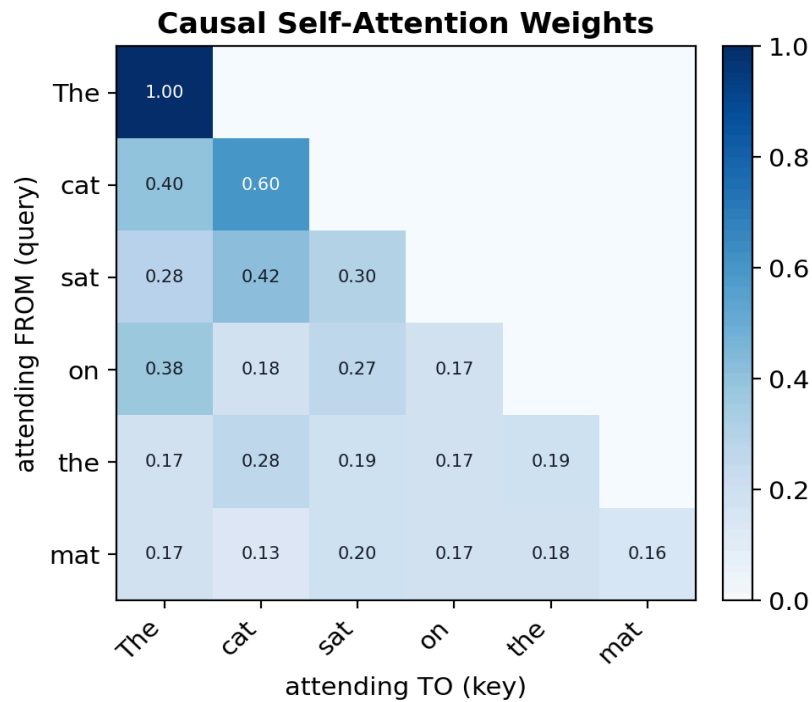
Attention alone has no notion of order — shuffling input tokens produces the same outputs, just reordered. Fix: add a fixed, unique, position-dependent vector to each token's embedding before attention.

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)}) \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{(2i/d)})$$

Sine/cosine at varying frequencies give every position a unique fingerprint. The exact formula doesn't need to be memorized — the role it plays does.

6. Causal Masking

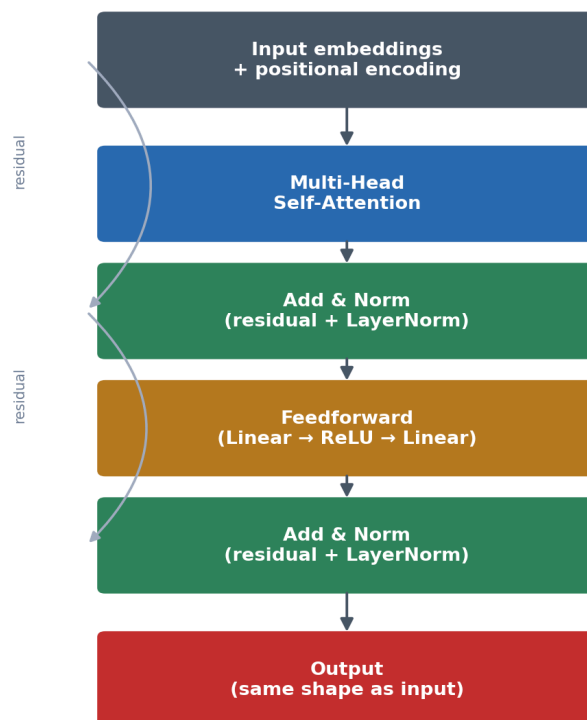
For autoregressive generation (GPT-style), a token may only attend to itself and earlier positions — future tokens don't exist yet at generation time. Implementation: set future positions to $-\infty$ BEFORE softmax (not after) — $\text{softmax}(-\infty)=0$, and rows still correctly sum to 1. Zeroing after softmax would break that normalization.



Row = query token, column = key token. Each row only has weight on itself and earlier tokens — the upper triangle is exactly zero.

7. The Full Transformer Block

$x \rightarrow \text{self-attention} \rightarrow \text{add \& norm} \rightarrow \text{feedforward} \rightarrow \text{add \& norm} \rightarrow \text{output}$. Residual connections ($x + \text{sublayer}(x)$) stabilize gradient flow through many stacked blocks; layer norm stabilizes activations per-token.



8. Quick Glossary

Term	Meaning
Q, K, V	Query/Key/Value — three learned linear projections of the input, used to compute attention.
d_k	Dimension of the Key vectors; used to scale attention scores.
Attention head	One independent Q/K/V attention computation; multi-head runs several in parallel.
Positional encoding	A fixed, position-dependent vector added to each token embedding so order is preserved.
Causal mask	Forces each token to only attend to itself and earlier tokens — required for autoregressive generation.
Residual connection	Adding a sublayer's input back to its output; stabilizes gradient flow through deep stacks.
Layer norm	Normalizes activations per-token across the feature dimension; stabilizes training.

9. The Big Picture

A Transformer = embeddings + positional encoding, fed through many stacked blocks like the one above. GPT stacks decoder blocks with causal masking (generates one token at a time). BERT stacks encoder blocks without masking (sees the whole sequence at once, for understanding tasks). Scale up the data and the number of stacked blocks, and this is the architecture behind every major modern LLM.